De l'invariant à l'interface.

Construire et expliquer LedgerX, étape par étape

Un carnet de construction pour comprendre les décisions, relier le code aux preuves et préparer une démonstration technique crédible.

LE FIL CONDUCTEUR

Un mouvement d'argent ne doit jamais exister à moitié.

Transactions atomiques, double entrée, verrous en base, idempotence et journal immuable forment le socle. L'interface vient rendre ce socle utilisable.

01 / COMPRENDRE

Audit de l'existant, périmètre et invariants.

02 / CONSTRUIRE

Transactions, sécurité, remboursements et contrôles.

03 / PROUVER

PostgreSQL réel, concurrence, mesures et limites.

04 / RENDRE UTILISABLE

React, montants exacts, parcours et documentation.

NOTE DE MÉTHODE

Le projet existait à la reprise. Ce document reconstruit l'ordre logique des travaux ; il ne prétend pas être un journal Git horodaté ni un travail sans assistance IA.

Comprendre avant d'étendre.

Étapes 01 et 02 · Partir du dépôt existant et préserver ses garanties financières.

01 Inspecter le projet existant

ACTION

Lecture du README, des routes FastAPI, services, modèles SQLAlchemy, migrations Alembic, scripts et tests. Repérage des limites de l'environnement et conservation du monolithe modulaire.

POURQUOI

Réécrire le dépôt aurait rendu plus difficile la préservation des invariants. Il fallait d'abord identifier qui ouvre la transaction, qui verrouille les comptes et qui fait respecter la comptabilité.

PREUVE / LIMITE

Repères : app/api, app/services, app/db, alembic/versions et tests. Le dépôt n'avait pas d'historique Git disponible pour attribuer chaque ligne antérieure.

02 Fixer les invariants de l'argent

ACTION

Montants en centimes entiers ; un transfert produit deux écritures opposées ; les comptes clients ne peuvent pas devenir négatifs ; le journal ne peut être ni modifié ni supprimé.

POURQUOI

Le solde est une vue matérialisée du journal. Les deux doivent rester cohérents. La base vérifie les contraintes financières même si une nouvelle route ou un autre client oublie un contrôle.

PREUVE / LIMITE

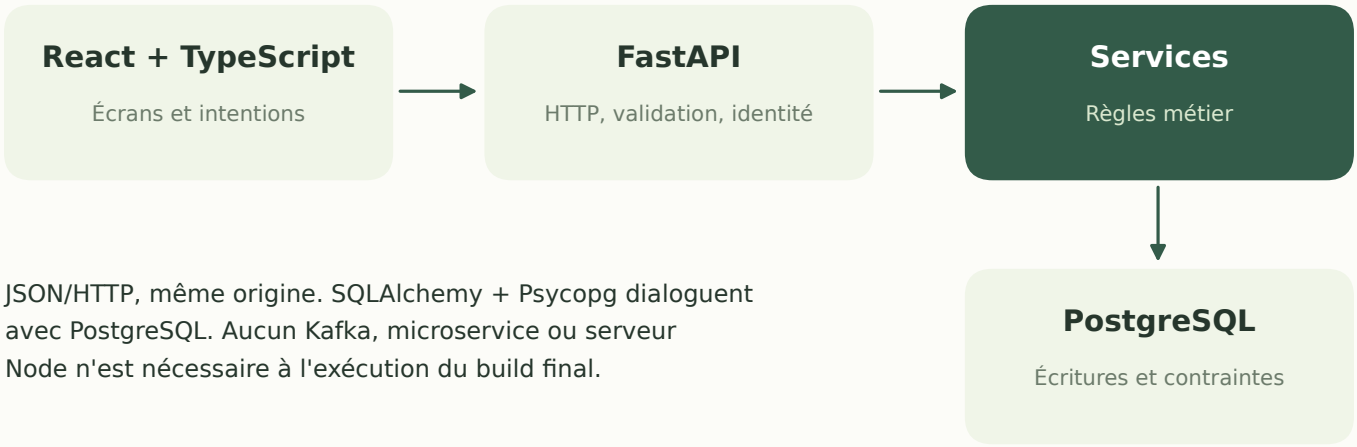
Compte système réservé au financement fictif. EUR uniquement. Plafond de 9 × 10^18 centimes. Contraintes, déclencheurs différés et tests d'intégrité dans PostgreSQL.

À SAVOIR EXPLIQUER

Pourquoi un float n'est pas adapté à l'argent ; pourquoi une validation Python seule ne protège pas tous les chemins d'écriture.

Un monolithe, des frontières.

Étapes 03 et 04 · Responsabilités séparées, transaction financière unique.



JSON/HTTP, même origine. SQLAlchemy + Psycopg dialoguent avec PostgreSQL. Aucun Kafka, microservice ou serveur Node n'est nécessaire à l'exécution du build final.

03 Conserver une séparation simple

ACTION

Les schémas Pydantic portent les contrats ; les routes gèrent HTTP et les transactions ; les services exécutent le métier ; les modèles et migrations définissent la persistance.

POURQUOI

L'OOP sert aux entités, sessions et contrats. Les services restent des fonctions explicites : aucune hiérarchie de classes artificielle n'est nécessaire pour déplacer de l'argent.

PREUVE / LIMITE

Repères : app/schemas, app/api, app/services et app/db. Le frontend appelle les mêmes routes documentées dans /docs.

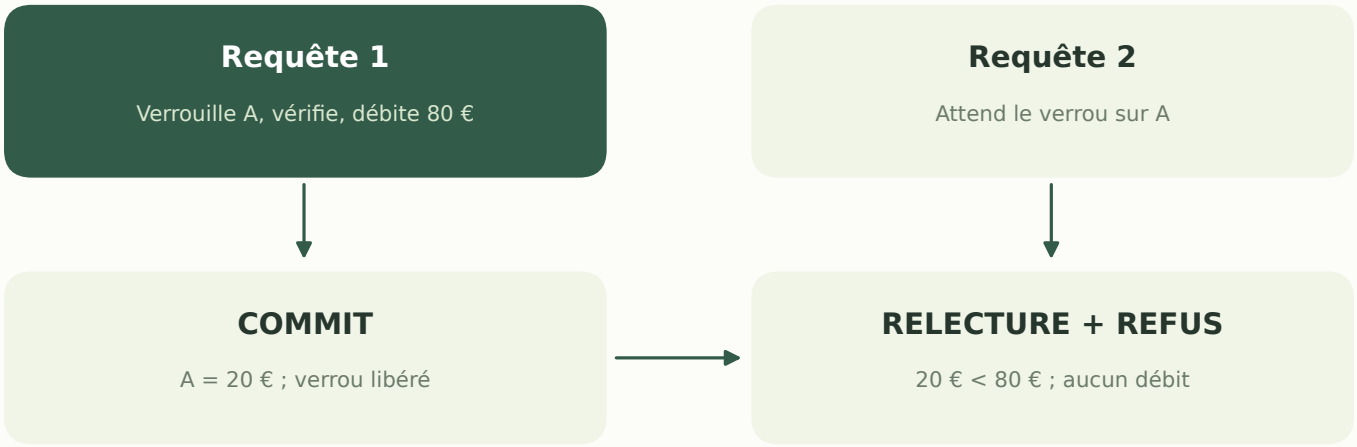
04 / UNE FRONTIÈRE ATOMIQUE

La route ouvre session.begin(). Le service réserve la clé, verrouille les comptes, vérifie les fonds, inscrit le transfert et ses deux écritures. Un seul commit rend tout visible ; toute exception annule l'ensemble.

Deux clients. Un même solde.

Étapes 05 et 06 · Traiter les courses avec les garanties de PostgreSQL.

EXEMPLE : A DISPOSE DE 100 € ; DEUX DEMANDES VEULENT DÉBITER 80 €



05 Verrouiller dans un ordre stable

ACTION

Les deux comptes sont verrouillés par UUID croissant avec SELECT FOR UPDATE. Tous les chemins financiers suivent le même ordre pour les comptes.

POURQUOI

Cela sérialise les dépenses sur une même source et réduit les risques d'interblocage entre virements opposés. Un mutex Python ne coordonnerait pas plusieurs processus API.

PREUVE / LIMITE

Tests de concurrence avec plusieurs connexions PostgreSQL. La sûreté n'élimine pas la contention : les virements partageant une source ralentissent sous forte concurrence.

06 / RAFRAÎCHIR AUSSI L'IDENTITY MAP

Un verrou SQL ne remplace pas automatiquement un objet déjà présent dans la Session SQLAlchemy. populate_existing=True recharge le solde verrouillé ; un test de régression protège ce cas subtil.

Une intention, un effet.

Étapes 07 et 08 · Répétition réseau, nouvelle opération et propriétaire.

07 Rendre les répétitions sûres

ACTION

La clé d'idempotence est indexée par utilisateur et type d'opération. Une empreinte SHA-256 du contenu détecte les réutilisations incompatibles. La réponse d'un transfert terminé est retrouvée en base.

POURQUOI

Une réponse HTTP perdue ne signifie pas un paiement annulé. Le client peut réémettre la même intention, avec le même contenu et la même clé, sans effectuer un second débit.

PREUVE / LIMITE

Optimisation introduite : INSERT ... RETURNING récupère directement une nouvelle réservation ; le chemin de conflit conserve sa lecture verrouillée. Ne pas attribuer les anciennes mesures à cette version sans la remesurer.

08 Ajouter identité et propriété

ACTION

Mots de passe dérivés avec scrypt et sel aléatoire ; tokens opaques aléatoires dont seul le condensat est conservé en base ; expiration et révocation. Chaque compte appartient à un utilisateur.

POURQUOI

L'authentification identifie ; l'autorisation décide. Connaître un UUID ou une clé d'idempotence ne doit pas permettre d'accéder aux comptes d'un autre utilisateur, même avec un rôle administrateur.

PREUVE / LIMITE

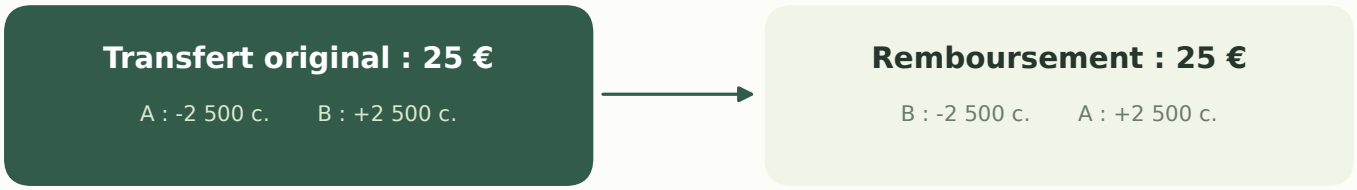
Tests d'absence de token, de tokens expirés et révoqués, de rôles et d'accès croisés. Le financement reste administratif et soumis à un flag explicite. Aucun MFA ni mécanisme de récupération de compte à ce stade.

LE POINT D'ENTRETIEN

Les tokens du navigateur restent en mémoire. Un rechargement déconnecte ; aucun bearer token n'est persisté dans localStorage.

Corriger sans effacer.

Étapes 09 et 10 · Compensation et contrôle indépendant du journal.



Lien reversal_of · les anciennes écritures restent intactes

09 Créer un transfert compensatoire

ACTION

Le destinataire d'origine peut retourner l'intégralité du paiement. La ligne d'origine est verrouillée pour sérialiser les demandes ; une contrainte unique interdit une seconde compensation.

POURQUOI

Une annulation destructive détruirait l'audit. Une nouvelle écriture conserve la cause et l'effet, vérifie les fonds disponibles et respecte encore la double entrée. Le verrou NO KEY UPDATE évite un conflit inutile avec les verrous de clés étrangères.

PREUVE / LIMITE

Remboursements complets uniquement, idempotents ; financement et remboursement d'un remboursement interdits. Tests de tentatives simultanées, de propriétaire et de solde insuffisant.

10 Recalculer indépendamment

ACTION

Le moteur de réconciliation somme les écritures de chaque compte et compare le résultat au solde stocké. Il vérifie aussi la structure des transferts et les totaux du journal dans un même instantané SQL.

POURQUOI

Le contrôle ne doit pas simplement réutiliser le résultat du service de virement : il confronte deux représentations indépendantes des montants.

PREUVE / LIMITE

Endpoint administratif ; rapport structuré ; aucune réparation silencieuse. Une corruption volontaire et isolée de sept centimes vérifie que l'écart est détecté.

Chercher les pannes utiles.

Étapes 11 et 12 · Observer sans divulguer, puis tester les garanties réelles.

11 Instrumenter les opérations

ACTION

Ajout de logs JSON avec identifiants de requête et de corrélation, métriques Prometheus et mesures de latence. Les erreurs SQL sont converties en réponses expurgées sans paramètres sensibles.

POURQUOI

On doit suivre un incident sans exposer mots de passe, tokens ou données financières dans les logs. Les labels de métriques sont bornés : pas d'identifiant de compte par série.

PREUVE / LIMITE

Injection de panne après modification en mémoire et après flush ; validation du rollback. Tests de lock timeout et de filtrage des exceptions inattendues. Timeouts de connexion, pool, verrou et requête explicités.

12 Tester sur PostgreSQL réel

ACTION

Tests unitaires, intégration avec plusieurs connexions, tests HTTP, migrations upgrade/downgrade lorsque réversibles et vérification de dérive ORM. Bases destructibles protégées par des suffixes dédiés.

POURQUOI

SQLite ne reproduit pas les verrous ni les déclencheurs PostgreSQL. Les tests financiers doivent vérifier aussi l'état final : soldes, écritures, comptes et idempotence après un échec.

PREUVE / LIMITE

Version 0004 validée : 147 tests unitaires/intégration et 2 E2E HTTP, 94,12 % de couverture. Rapports archivés dans artifacts/releases. Le premier run GitHub Actions de d65fb3c a réussi ; la PR #1 a aussi été confirmée verte par le propriétaire. La mise à jour du workflow attend son run.

RÈGLE DE PRÉSENTATION

Un test écrit n'est pas un test exécuté. Une couverture élevée n'est pas une preuve d'absence de bugs ; elle complète les assertions métier.

Mesurer avant de promettre.

Étapes 13 et 14 · Référence du 20 octobre ; résultats de la version 0004 en page 14.

13 Construire une matrice de charge

ACTION

Dix scénarios HTTP, 1, 8 et 32 clients en concurrence, trois répétitions : 90 cas et 18 000 requêtes chronométrées. Les scénarios distinguent lectures, virements indépendants, source partagée, répétitions et remboursements.

POURQUOI

Une moyenne globale masquerait l'attente des verrous et les queues de latence. On conserve les données brutes, les percentiles et les contrôles de cohérence après chaque scénario.

PREUVE / LIMITE

Référence locale : tous les cas avaient les réponses attendues et une réconciliation cohérente. Les réponses d'insuffisance de fonds attendues sont des succès du scénario de test, pas des virements réussis.

Scénario / 8 clients	Requêtes/s	p95
Lecture du solde	926,5	10,44 ms
Virements indépendants	443,1	21,44 ms
Source partagée	365,5	56,52 ms

Médianes des 3 répétitions, y compris médiane des p95 ; pas un p95 agrégé.

14 Profiler et conserver la référence

ACTION

cProfile, comptage SQL, EXPLAIN ANALYZE et notebook avec courbes. La référence est archivée avant l'optimisation de la réservation d'idempotence, qui vise à supprimer une lecture SQL redondante.

POURQUOI

La profondeur du journal, la pagination et le niveau de concurrence n'ont pas les mêmes coûts. Un plan SQL plus rapide ne prouve pas automatiquement une amélioration HTTP de bout en bout.

PREUVE / LIMITE

Référence : 7 appels SQL par transfert ; 6 observés sur la version 0004. Ces comptes excluent les instructions internes aux triggers. 32 clients testés ne donnent pas une capacité commerciale garantie.

Rendre le système utilisable.

Étapes 15 et 16 · Construire une interface originale sur les contrats existants.

15 Ajouter React sans refaire l'API

ACTION

React et TypeScript construisent les écrans ; Vite produit un bundle statique servi par FastAPI. Deux lectures protégées sont ajoutées : /auth/me et la liste paginée /accounts, filtrée par propriétaire.

POURQUOI

Le dashboard utilise les comptes réels et leurs écritures. Les graphiques indiquent leur fenêtre et leur périmètre chargé ; aucune fausse dépense ou banque externe n'est injectée dans le produit.

PREUVE / LIMITE

Connexion, création de compte, réception par UUID, virement avec confirmation, historique, détail, remboursement et réconciliation admin. CSS responsive original, dialogue natif, navigation clavier et réduction des animations.

16 Préserver la précision et l'intention

ACTION

Les centimes restent des BigInt dans le navigateur. lossless-json préserve les entiers au passage JSON. Une intention de paiement fige la charge utile et la clé d'idempotence pour les nouvelles tentatives.

POURQUOI

JSON.parse avec Number peut arrondir un grand entier avant même son affichage. Une panne réseau peut survenir après le commit : le bouton Réessayer doit réutiliser l'intention au lieu d'en créer une autre.

PREUVE / LIMITE

26 tests frontend exécutés à cette étape couvrent conversions, bornes et reprises. La compilation TypeScript + Vite réussit. Les tests navigateur sont distingués entre API simulée et parcours sur PostgreSQL réel.

LIMITES ASSUMÉES

La session et la clé en attente restent en mémoire. Après rechargement ou expiration, vérifier l'historique avant de créer un nouveau paiement.

Pouvoir le refaire. Et l'expliquer.

Étapes 17 et 18 · Reproductibilité locale et présentation honnête des résultats.

17 Rendre l'installation reproductible

ACTION

Scripts dédiés pour Node local, npm, compilation, tests navigateur et vérification backend. uv.lock et package-lock.json enregistrent les dépendances. Caches et téléchargements restent dans le projet.

POURQUOI

Un reviewer doit pouvoir suivre quelques commandes, sans compte cloud, et comprendre ce qui démarre. FastAPI sert le frontend compilé ; Node n'est nécessaire qu'au build et au développement frontend.

PREUVE / LIMITE

scripts/verify_local.sh utilise _test et _bench ; scripts/verify_web.sh utilise _bench.
Le README précise chaque commande. Ne pas commettre node_modules, .venv, .runtime ni les secrets.

18 Préparer une démo vérifiable

ACTION

Démontrer deux comptes, un financement fictif administrateur, un virement, un remboursement et une réconciliation. Relier chaque écran aux services et à un test pertinent, puis présenter les mesures archivées.

POURQUOI

Le projet est convaincant lorsqu'on sait expliquer un compromis, reproduire un résultat et reconnaître une limite. Ajouter des infrastructures ou des features décoratives ne remplace pas cette maîtrise.

PREUVE / LIMITE

Aucun argent réel, aucune conformité bancaire revendiquée. TLS public, sauvegarde/restauration, quotas, protection distribuée, surveillance et charge longue restent à traiter avant une ouverture à de vrais utilisateurs.

PROVENANCE ET NOM

Travail accompagné par IA ; revue et compréhension personnelles nécessaires. LedgerX est un nom de travail déjà utilisé ailleurs ; le projet est indépendant et non affilié à Revolut ou à Ledger.

Du récit aux preuves.

Les fichiers à consulter pour vérifier les explications et reproduire les parcours.

01 Architecture et décisions
README.md · docs/ENGINEERING_REPORT.md

02 Invariants et schéma
app/db/models.py · alembic/versions/

03 Transactions et verrous
app/services/transfer_service.py

04 Idempotence et identité
app/services/idempotency_service.py · auth_service.py

05 Réconciliation
app/services/reconciliation_service.py

06 Preuves PostgreSQL
tests/ · artifacts/integration.xml · artifacts/e2e.xml

07 Mesures et référence
artifacts/latency/ · artifacts/baseline/

08 Analyse visuelle
analysis/ledgerx_performance.ipynb

09 Interface et contrats
frontend/src/ · frontend/tests/ · frontend/e2e/

10 Résultats navigateur
artifacts/frontend-browser-results.json

11 Guide de démonstration
docs/DEMO.md

ÉTAT DU RAPPORT AU MOMENT DE LA GÉNÉRATION

Version 0004 vérifiée : 12 réussis, 0 en échec, 0 ignorés. Deux parcours réels dans Docker ; déploiement local terminé le 22 octobre.

De la démo locale au web.

Étapes 19 et 20 · Docker local vérifié sur 0004 ; tunnel public testé manuellement.



Accès public futur : tunnel HTTPS ou hébergement distant.

19 Borner le coût de l'authentification

ACTION

Connexions et inscriptions partagent une fenêtre glissante et deux places simultanées. Une deque, des compteurs et un verrou protègent les réservations en mémoire.

POURQUOI

Le hachage scrypt coûte du CPU et de la mémoire. Une requête refusée reçoit HTTP 429 et Retry-After avant la vérification du mot de passe.

PREUVE / LIMITE

Six tests dédiés passent. Limites locales au processus, remises à zéro au redémarrage ; plusieurs visiteurs derrière un proxy peuvent partager le même quota.

20 Préparer et vérifier le conteneur public

ACTION

Dépendances Python verrouillées, PORT fourni par l'hébergeur, arrêt si migration en échec. La dernière étape Docker produit l'application, pas l'image de tests.

POURQUOI

La préparation distingue la base locale de la base distante. Le scénario Docker isolé transfère puis rembourse entre deux vrais profils clients.

PREUVE / LIMITE

Version 0004 : 149 tests backend, 26 frontend et 12 navigateur passent. Sauvegarde créée et app locale migrée. Restauration non encore testée.

HISTORIQUE GITHUB

Importer honnêtement le travail local. Depuis la publication, README corrigé en PR #1 ; les vérifications ont été confirmées par le propriétaire.

Retirer le coût de l'historique.

Étapes 21 et 22 · Projection incrémentale et charge à arrivées fixes.



21 Maintenir le total du journal à chaque insertion

ACTION

Le trigger additionne les nouvelles écritures au lieu de relire tout l'historique. Il protège la projection ; les contraintes de double entrée restent.

POURQUOI

Le coût de validation ne doit pas augmenter avec l'âge du compte. La réconciliation recalcule encore le journal complet indépendamment.

PREUVE / LIMITE

Dix tests PostgreSQL ajoutés : falsification, lots, concurrence, rollback et migration avec fonds. Tous passent dans la suite PostgreSQL sur le Mac.

22 Mesurer la latence sans masquer la saturation

ACTION

Arrivées programmées à débit fixé, tâches simultanées bornées. Chaque perte est comptée ; le délai de programmation entre dans la latence.

POURQUOI

En boucle fermée, un serveur lent ralentit aussi le générateur. La nouvelle mesure distingue débit offert, réponses, pertes, p95 et p99.

PREUVE / LIMITE

18 cas mesurés : comptabilité cohérente, objectifs de charge élevés non tenus. Les graphiques exposent les abandons du générateur, sans les masquer.

MESURE ET COMPROMIS

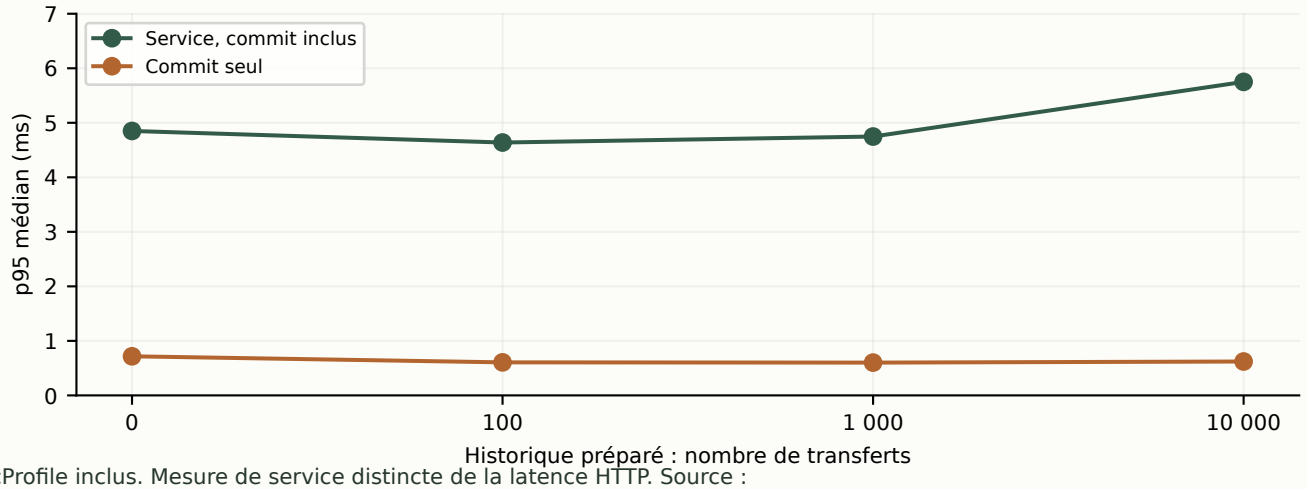
Projection : mises à jour et WAL supplémentaires. Verrous toujours nécessaires. Les anciens chiffres restent une référence archivée, pas une preuve de gain.

Les preuves du 22 octobre.

Étapes 23 et 24 · Mesures réelles, limites explicites et app disponible sur le Mac.

149	26	12
tests backend	tests frontend	tests navigateur

PROFILAGE : 3 RÉPÉTITIONS DE 100 TRANSFERTS PAR PROFONDEUR



cProfile inclus. Mesure de service distincte de la latence HTTP. Source : artifacts/latency/profiling.json.

23 / CONSERVER LES LIMITES DE CHARGE

30 000 arrivées proposées : 18 439 envoyées, 11 561 abandonnées par le générateur. Aucune réponse inattendue ; rapprochements cohérents. Les six essais courts à 100 req/s passent, mais seulement deux sur six à 300 et aucun à 600. Aucun SLO garanti.

24 / LIVRER ET PARTAGER

Image Docker construite, base personnelle sauvegardée, migration 0004 et contrôles de disponibilité réussis. Utilisation : http://127.0.0.1:8000. Le tunnel HTTPS temporaire permet à un ami de s'inscrire pendant que le Mac reste allumé ; pas besoin de compte Cloudflare.

CE QUI RESTE À VÉRIFIER

CI du premier push et PR #1 confirmées vertes par le propriétaire. Restauration de sauvegarde sur base isolée, charge longue et hébergement cloud restent à faire.

Partir du code et le lancer.

Guide pratique pour debuter : ces etapes repartent du depot publie, pas d'un dossier vide.

01 / RECUPERER ET PREPARER

TERMINAL - A LA RACINE DU PROJET

```
git clone https://github.com/anaselmadhi/LedgerX.git
cd LedgerX
cp .env.example .env
mkdir -p .runtime/tmp .runtime/cache/uv
TMPDIR="$PWD/.runtime/tmp" UV_CACHE_DIR="$PWD/.runtime/cache/uv" \
  UV_PYTHON_DOWNLOADS=never uv sync --frozen --python 3.12
```

Prerequis Mac : Git, Python 3.12, Docker Desktop (installe puis ouvert) et une connexion internet. Si uv manque : brew install uv. Le depot contient le code, les migrations, les tests et la configuration. .env est local et ignore par Git : ne publie jamais son contenu.

02 / ORDRE DE CONSTRUCTION DU LOGICIEL

01	**Backend**	Routes FastAPI -> schemas Pydantic -> services -> PostgreSQL via SQLAlchemy.
02	**Comptabilite**	Montants en centimes, deux ecritures par transfert, contraintes DB et migrations Alembic.
03	**Tests**	Tests unitaires puis PostgreSQL reel; verrous et transactions ne se prouvent pas avec SQLite.
04	**Interface**	React/TypeScript; Node construit les fichiers web, FastAPI les sert ensuite.
05	**Livraison**	Docker empaquette API + interface; Compose lance aussi PostgreSQL; Actions verifie les changements.

CHOIX PYTHON / C++

Garder Python pour ce backend : les mesures pointent vers l'attente PostgreSQL et les verrous. C++ ne les supprime pas; l'envisager pour un composant CPU-mesure ou un projet moteur d'ordres distinct.

REFAIRE SEULE

Lis les pages 02 a 14 pour les decisions de conception, puis suis les commandes des pages 15 a 20 pour les reproduire.

Image, conteneur, donnees.

Docker emballe et lance le programme. Il ne remplace pas PostgreSQL et ne publie pas a lui seul un site sur internet.

Image

Modele construit depuis Dockerfile : Python, dependances, code et interface compilee.

Conteneur

Programme en execution depuis une image. Compose lance app et db ensemble.

Volume

Le dossier .runtime/postgres conserve les donnees PostgreSQL quand le conteneur est arrete.

Port

127.0.0.1:8000 rend le site accessible sur ce Mac seulement. PostgreSQL reste local aussi.

COMMANDES COURANTES

TERMINAL - TOUS LES JOURS

```
docker compose up -d db app         # demarrer
docker compose ps                   # etat (healthy = prete)
docker compose logs --tail=100 app  # erreurs de l'app
docker compose stop app db          # arreter sans supprimer
docker compose start db app         # redemarrer
```

PREPARER L'INTERFACE ET L'IMAGE

```
bash scripts/setup_frontend.sh      # installe Node 24 local
bash scripts/frontend.sh install   # dependances verrouillees
bash scripts/frontend.sh build     # compile vers app/web
docker compose build app           # reconstruit l'image API + UI
```

DONNEES PERSONNELLES

Ne supprime pas le volume pour depanner. La base contient profils et mouvements; les sauvegardes restent privees dans .runtime/backups/.

Un robot verifie chaque changement.

CI signifie integration continue : GitHub cree une machine temporaire et rejoue les controles apres un push ou l'ouverture d'une PR.

1. Declencheur push ou pull_request : chaque changement propose est verifie.

2. Machine ubuntu-24.04; environnement vierge, independant du Mac.

3. Outils checkout@v7 recupere le code; setup-python@v7 installe Python 3.12; setup-node@v7 installe Node 24; uv sync suit uv.lock.

4. Base jetable Un service PostgreSQL 16 est cree pour les tests. La base personnelle n'est pas utilisee.

5. Verification Ruff, format, tests Python/PostgreSQL, migrations, HTTP, frontend et navigateur.

6. Docker Un second job construit l'image de production et execute deux parcours navigateur avec PostgreSQL.

7. Resultats upload-artifact@v7 archive les rapports. Vert = jobs passes pour ce commit; cela ne prouve ni capacite maximale ni cloud.

CE QUE LA CI NE FAIT PAS

Elle n'heberge pas le site et ne touche pas a la base du Mac. Le workflow verifie et archive des rapports; il ne deploie rien.

Publier sans inventer l'historique.

Le depot a d'abord ete importe depuis le Mac. Chaque evolution suivante se fait sur une branche avec un changement reel, puis une pull request.

PREMIERE PUBLICATION - DEJA FAITE, NE PAS REJOUER

TERMINAL - PUBLICATION INITIALE REELLE

```
git init -b main                # si aucun depot Git local
git add .github .gitignore .dockerignore .env.example app alembic frontend \
  scripts tests docker analysis docs output/pdf README.md INTERVIEW_NOTES.md \
  pyproject.toml uv.lock alembic.ini Dockerfile compose.yaml compose.share.yaml render.yaml
git add artifacts/README.md artifacts/benchmark.json artifacts/profiling.json \
  artifacts/baseline artifacts/figures artifacts/latency artifacts/releases \
  artifacts/frontend-delivery.json  # uniquement les preuves relues
git diff --cached --check       # espaces parasites ?
git commit -m "Import LedgerX financial demo developed locally"
git remote add origin https://github.com/anaselmadhi/LedgerX.git
git fetch origin
git merge origin/main --allow-unrelated-histories --no-edit -X ours
git push -u origin main
```

CHAQUE CHANGEMENT SUIVANT - EXEMPLE

TERMINAL PUIS PAGE GITHUB

```
git switch main && git pull --ff-only
git switch -c docs/ci-and-rebuild-guide
# modifier puis relire le diff
git diff --check && git diff
git add .github/workflows/ci.yml analysis/build_journey_pdf.py \
  docs/ENGINEERING_REPORT.md output/pdf/ledgerx-construction-pas-a-pas.pdf
git commit -m "Modernize CI and document LedgerX rebuild"
git push -u origin docs/ci-and-rebuild-guide
# GitHub: Compare & pull request -> checks -> Merge
```

Une PR compare la branche a main : elle n'est pas vide si les changements ont ete pousses. Le mot de passe HTTPS est un token GitHub, pas le mot de passe normal du compte. Ne partage jamais ce token.

ETAT AU 27 OCTOBRE

Import publie; PR #1 (README et captures) fusionnee, checks confirmes verts par le proprietaire. La branche docs/ci-and-rebuild-guide contient la modernisation CI; verifier les checks de sa PR avant fusion.

Le parcours reproductible.

La verification normale suffit pour controler le logiciel. Les mesures longues sont facultatives et ne sont pas necessaires pour ouvrir l'application chaque jour.

A / VERIFIER LE CODE ET POSTGRESQL

TERMINAL - TESTS ET MIGRATIONS

```
bash scripts/verify_local.sh
```

Utilise ledgerx_test (jetable) pour les tests destructifs et ledgerx_bench pour les parcours HTTP. Ne vide pas ledgerx, la base personnelle. Lance le pipeline seul.

B / VERIFIER L'IMAGE DOCKER ET LE NAVIGATEUR

TERMINAL - DOCKER DE TEST ISOLE

```
bash scripts/verify_web.sh --container
```

Utilise ledgerx_bench, demarre une API temporaire sur le port 8002, puis joue les parcours navigateur contre le vrai API et PostgreSQL. Ne lance pas de benchmark concurrent.

C / METTRE A JOUR L'APPLICATION PERSONNELLE

PIPELINE LONG - PAS POUR CHAQUE DEMARRAGE

```
ENABLE_DEV_FUNDING=true bash scripts/release_local.sh --full-profile
```

Enchaine tests, migrations, mesures, navigateur Docker, build, arret de l'app, pg_dump prive, migrations et smoke check. Les profils existants sont conserves. N'interromps pas pendant sauvegarde/migrations.

USAGE QUOTIDIEN APRES UN RELEASE VALIDE

```
docker compose start db app
# ouvrir http://127.0.0.1:8000
```

RESULTATS REELS

Dernier release local : 149 tests backend, 26 frontend et 12 navigateur passes. En charge elevee, le generateur a aussi abandonne des arrivees; aucune capacite maximum n'est certifiee.

Local, tunnel ou cloud.

Ces choix changent qui peut joindre le serveur, combien de temps il reste disponible et ou les donnees sont stockees.

PARTAGER TEMPORAIREMENT DEPUIS LE MAC

LE LIEN HTTPS EST AFFICHE DANS LE TERMINAL

```
docker compose up -d db app
bash scripts/share_local.sh
```

Envoie le lien trycloudflare a ton ami. Garde Mac, Docker et terminal du tunnel allumes; Ctrl+C coupe le lien public. L'ami cree un profil client normal. L'admin peut ajouter des fonds fictifs si ENABLE_DEV_FUNDING=true dans le .env local. Les donnees restent sur PostgreSQL de ton Mac.

HEBERGEMENT CLOUD - OPTIONNEL, PAS ENCORE FAIT

Le depot prepare Render (render.yaml) et decrit le parcours dans docs/DEPLOYMENT.md. En bref : creer un PostgreSQL distant dedie (Neon), garder son URL comme secret Render, creer le service web depuis le depot, attendre migrations et health check, creer l'admin distant separement, puis verifier l'URL publique. Ne jamais utiliser l'URL de la base locale.

APRES AVOIR CONFIGURE LES SERVICES DISTANTS

```
.venv/bin/python -m scripts.create_remote_admin
.venv/bin/python -m scripts.smoke_public --url https://TON-SERVICE.onrender.com
```

Le service gratuit peut s'endormir et la base gratuite Render expire apres 30 jours. Kubernetes n'est pas requis. Ce projet n'a pas ete deploye sur le cloud.

RECUPERATION A COMPLETER

Tester pg_restore dans une base separee, verifier le schema et lancer la reconciliation. Ne restaure jamais sur ledgerx pendant ce test. La CI du changement de workflow doit aussi passer avant fusion.

A quoi sert chaque dossier ?

La requete HTTP traverse des couches courtes. Chaque fichier a une responsabilite; la transaction financiere reste dans PostgreSQL.

app/api/ + main.py + start.py	Routes HTTP, auth, limites, middleware, demarrage et serving du frontend.
app/schemas/	DTO Pydantic : formes d'entree/sortie, validation et limites; pas de requetes SQL.
app/services/	Regles metier : comptes, transferts, cles idempotentes et audit; le routeur ouvre/termine la transaction.
app/db/	Modeles SQLAlchemy, moteur et fabrique de sessions; une session par requete, jamais partagee entre threads.
app/core/	Configuration, montants/cursors, securite, erreurs, rate limit et observabilite.
alembic/	Historique du schema; 0001-0004 ajoutent progressivement contraintes, auth, remboursements et projection.
frontend/src/	App React; components = ecrans; lib/api.ts = HTTP; money.ts = cents/BigInt; types.ts = contrats.
frontend/tests/ + e2e/	Vitest teste API/montants; Playwright teste desktop, mobile et le vrai parcours HTTP.
tests/	Unitaires, PostgreSQL, course, rollback, migration, securite, endpoints et scenarios de charge.
scripts/	Installer, lancer, verifier, mesurer, sauvegarder, partager et preparer un admin distant.
compose.yaml + Dockerfile + docker/	Services app/db/test; image web-build/runtime/test; initialisation des bases jetables.
.github/ + docs/	Workflow CI et modele PR; guides engineering, usage local, partage, latency et deploiement.
analysis/ + artifacts/	Analyse/figures/PDF generator; preuves de runs, mesures et rapports; artefacts distingues par revision.
.runtime/ + app/web/	Etat local ignore : DB, caches, sauvegardes. app/web est un build genere, pas la source React.

TRACE D'UNE REQUETE

frontend/src/lib/api.ts -> app/api -> schema -> service -> SQLAlchemy/Psycopg -> PostgreSQL. La reponse HTTP est envoyee apres commit.

Pourquoi ces choix ?

Les complexites donnent le travail attendu, pas une latence garantie. Le choix vise LedgerX; aucun algorithme n'est le meilleur partout.

Centimes entiers + BigInt

Python int et PostgreSQL BIGINT font des additions exactes. Le navigateur garde BigInt et lossless-json evite l'arrondi de Number. Cout constant par operation; limite explicite de montant.

Journal double entree

Deux lignes signees par transfert; somme zero. Un journal append-only rend l'audit possible. Le solde courant est une projection pour les lectures rapides, controlee contre les ecritures.

Projection incrementale

Trigger groupe seulement les nouvelles lignes et applique les deltas. La validation ne rescane plus E_a ecritures historiques; echanges : updates, WAL et stockage supplementaires.

Verrous PostgreSQL ordonnes

Trier les deux UUID puis FOR UPDATE : deux chemins A->B/B->A prennent le meme ordre. Cout O(1) pour deux comptes; la base bloque le meme solde jusqu'au commit.

Identite SQLAlchemy rafraichie

populate_existing=True remplace une entite potentiellement vieille dans l'identity map apres le verrou. FOR UPDATE seul ne suffit pas si Python garde un ancien solde.

Idempotence persistante

Cle composite (principal, operation, cle) indexee B-tree; SHA-256 du JSON canonique reconnait le payload. INSERT ON CONFLICT RETURNING evite une lecture pour une nouvelle cle.

Curseur + index compose

(account_id, created_at, id) permet un seek B-tree O(log E + L). OFFSET saute O lignes et coute environ O(log E + O + L); curseur lie au compte, pas un jeton d'autorisation.

Deque du rate limiter

Deque de temps, compteurs et verrou bref donnent un cout amorti proche de O(1) pour expirer/admettre. Hachage scrypt/SQL restent hors verrou. Etat local au processus.

RECONCILIATION = AUDIT, PAS HOT PATH

Une requete SQL independante recalcule les sommes depuis toutes les ecritures et compare balances + projection. Travail proportionnel a l'historique; executee a la demande, sans reparer silencieusement.

Mesurer le bon niveau.

Trois mesures differentes repondent a trois questions differentes : API complete, service/commit, et plan d'une requete SQL.

HTTP ferme - scripts/benchmark.py

ThreadPoolExecutor envoie une requete par client a la fois. Vrai TCP vers API + PostgreSQL; login/setup hors chrono; percentiles, statuts, soldes et reconciliation en sortie.

Service/commit - profile_workload.py

cProfile observe les fonctions Python; SQLAlchemy compte execute/duree; perf_counter separe service et commit. 100 transferts par profondeur, plusieurs repetitions.

Arrivees fixes - benchmark_open_loop.py

asyncio planifie les requetes a l'avance sans attendre chaque reponse. Compte les demandes proposees, envoyees et abandonnees; latence inclut attente de dispatch.

COMMANDES ET OUTILS

```
docker compose up -d db              # lancer PostgreSQL local
# mesures isolees dans ledgerx_bench
bash scripts/verify_latency.sh          # matrice complete
bash scripts/verify_latency.sh --quick  # fumee courte, pas une reference
# rapports JSON/Markdown/PNG dans artifacts/latency/
# EXPLAIN (ANALYZE, BUFFERS) compare les plans offset/cursor
```

Le profil courant utilise 0, 100, 1 000 et 10 000 transferts prepares, 100 echantillons et 3 repetitions. cProfile ajoute du cout : ses ms ne sont pas la latence HTTP. Le commit inclut les triggers differes; le chrono SQL ne compte pas leur SQL interne.

RESULTAT 0004

p95 service median : 4,85 ms a profondeur 0, 5,75 ms a 10 000. Cela montre la validation independante de l'historique dans ce test; ce n'est pas un gain causal HTTP ni un SLO.

Role, raison, usage.

Chaque outil repond a un besoin precis. Les lignes de commande sont celles du depot; verrouiller les versions rend les reprises comparables.

Python 3.12 + FastAPI + Uvicorn

Règles métier et routes HTTP. Pydantic valide les entrées. Routes synchrones pour Psycopg bloquant; Uvicorn sert l'ASGI.

SQLAlchemy 2 + Psycopg 3

ORM/session et pilote PostgreSQL. Session par requete, transaction explicite dans la route; service flush, ne commit pas.

PostgreSQL 16 + Alembic

Source durable, index, contraintes, verrous et triggers. Alembic versionne les changements; migrations vérifiées sur _test.

React 19 + TypeScript + Vite

Interface composants/types; Vite compile statique dans app/web. Node sert au build, pas au runtime de production.

BigInt + lossless-json

Transporte les cents au-delà de Number.MAX_SAFE_INTEGER sans arrondi. Ne convertir en Number que des ratios graphiques bornés.

pytest + Ruff + Vitest

pytest couvre unitaires/PostgreSQL/HTTP; Ruff vérifie Python; Vitest vérifie montants et client API.

Playwright + HTTPX

Playwright contrôle l'interface desktop/mobile; HTTPX pilote API, vrais parcours et matrices de charge.

Docker + Compose

Image reproductible et services app/db/test/bench. Volume local garde la DB; ports app et DB liés à localhost.

GitHub Actions

Machine temporaire: Postgres de test, deux jobs, rapports. Le workflow actuel ne publie pas l'app.

Prometheus + logs JSON

Compteurs et histogrammes par route/type de resultat; requetes loguees sans secret ni ID arbitraire.

uv + lockfiles

uv.lock fige Python; package-lock.json fige npm; uv sync --frozen et npm ci reproduisent le graphe.

cProfile + EXPLAIN

Le premier trouve les fonctions CPU; le second montre le plan SQL execute. Comparer avec HTTP reel avant d'annoncer un gain.

NE PAS AJOUTER POUR FAIRE JOLI

Pas de microservices, Redis, Kafka ou Kubernetes : une transaction PostgreSQL unique rend le flux plus simple a raisonner. Ajouter une brique seulement si une mesure ou un besoin reel le justifie.

Les reponses a defendre.

Pour chaque decision : invariant, risque, mecanisme, preuve, compromis. C'est le fil conducteur a utiliser en entretien.

Pourquoi des centimes entiers ?

Pas d'arrondi binaire. Un transfert de 12,34 EUR vaut 1 234; l'API refuse fraction, zero et signe negatif.

Pourquoi double entree ?

Chaque debit a son credit; somme globale nulle. Le journal immuable garde l'audit et la reconciliation peut recalculer les soldes.

Pourquoi la DB decide ?

Le verrou FOR UPDATE et les contraintes couvrent plusieurs threads/processus. Un mutex Python ou un cache seul ne protege pas les autres workers.

Pourquoi idempotence persistante ?

Le serveur peut commit puis perdre la reponse. Meme cle + meme payload retrouve le transfert; cle reutilisee avec payload different donne conflit.

Pourquoi une projection ?

La validation 0003 rescannait les ecritures du compte. La projection incrementale rend cette validation independante de E_a; reconciliation continue le scan complet.

Pourquoi un monolithe modulaire ?

Debit, credit, journal et cle se commitent dans une seule DB. Des services separes imposeraient outbox/sagas sans benefice prouve ici.

Pourquoi pas C++ pour LedgerX ?

Profilage: attente DB/verrous, pas preuve que le bytecode est le goulet. C++ ne rend pas paralleles deux debits sur le meme compte; bon sujet pour un moteur d'ordres distinct.

Pourquoi pas cloud / Kubernetes ?

Le besoin actuel est portfolio/demo locale. Docker reproduit l'execution; GitHub Actions verifie. Hebergement, sauvegarde restauree et haute dispo restent des travaux distincts.

COMMENT PARLER DES MESURES

Distinguer plan SQL, service avec profiler, HTTP ferme et arrivees fixes. Dire aussi ce qui n'a pas passe: 11 561 arrives sur 30 000 ont ete abandonnees par le generateur a charge elevee.